

MODUL DEPLOY VISUALISASI DATA MENGUNAKAN STREAMLIT

Mentor: Prof. Dr. Enny Itje Sela, S.Si., M.Kom.



Disusun oleh :

5210411144 – Muhammad Izza Iqbal

**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS & TEKNOLOGI
UNIVERSITAS TEKNOLOGI YOGYAKARTA**

BAB 1 Pengenalan Streamlit

Pada materi sebelumnya, kita telah membahas sedikit tentang Dashboard. Sekarang saatnya kita berkenalan dengan tool yang akan kita gunakan untuk membuat dashboard. Tentunya Anda sudah bisa menebak nama tool tersebut, bukan?

Yap, betul. Pada materi ini kita akan berkenalan dengan Streamlit.



Streamlit merupakan salah satu *open-source web app framework* untuk bahasa pemrograman Python. Ia memungkinkan kita membuat web app yang baik dan interaktif dalam waktu yang singkat. Selain itu, ia juga kompatibel dengan berbagai library populer seperti NumPy, pandas, matplotlib, dll. Inilah yang menjadi alasan streamlit sering digunakan oleh para praktisi data dan machine learning.

Apakah Anda sudah mulai tertarik untuk belajar streamlit? Yuk, kita mulai dengan menginstalnya!

Menginstal Streamlit

Seperti biasa, sebelum mulai menggunakan streamlit tentunya kita perlu menyiapkan *environment*-nya terlebih dahulu. Berikut merupakan tahapan untuk melakukannya.

1. Untuk menggunakan streamlit, kita perlu menulis kode Python dalam berkas **.py**. Oleh karena itu, Anda perlu menginstal code editor,

seperti [Visual Studio Code](#), [Sublime Text](#), atau [PyCharm](#) untuk mempermudah dalam menulis kode.

2. Selanjutnya, Anda perlu mengaktifkan virtual environment sebelumnya yang telah kita gunakan dalam materi latihan. Aktifkan virtual environment dengan menjalankan perintah berikut.

1. `conda activate main-ds`

3. Tahap berikutnya adalah menginstal streamlit. Berikut merupakan perintah yang bisa Anda gunakan untuk menginstal streamlit.

1. `pip install streamlit`

Membuat "Hello, world!" App dengan Streamlit

Selamat, pada tahap ini Anda telah berhasil menginstal streamlit. Untuk memastikan proses instalasinya berjalan dengan lancar, kita akan membuat sebuah "Hello, world!" app dengan streamlit.

Berikut merupakan tahapan dalam membuat "Hello, word!" app dengan streamlit.

1. Buatlah sebuah folder baru bernama **hello-world**.
2. Bukalah folder tersebut dengan code editor favorit Anda.
3. Berikutnya, buatlah sebuah berkas python dengan nama **hello-world.py**.
4. Buka berkas python tersebut, lalu tambahkan kode berikut.

```
1. import streamlit as st
2.
3. st.write(
4.     """
5.     # My first app
6.     Hello, para calon praktisi data masa depan!
7.     """
```

8.)

5. Untuk menjalankan kode tersebut, Anda perlu membuka terminal, command prompt atau powershell. Arahkan path-nya ke dalam folder **hello-world** yang sebelumnya telah dibuat. Lalu, jalankan perintah berikut.

1. `streamlit run hello-world.py`

Jika seluruh proses di atas berjalan dengan lancar, Anda akan menemukan tampilan web app seperti berikut.

My first app

Hello, para calon praktisi data masa depan!

Yey, selamat! Anda berhasil membuat web app pertama menggunakan streamlit. Bagaimana menurut Anda? Mudah, bukan?

Nah, pada materi berikutnya, kita akan melihat beberapa basic element dalam streamlit. Apakah Anda sudah siap? Yuk, kita lanjut!

BAB 2 Basic Element dalam Streamlit

Sebelumnya, Anda telah berhasil menginstal dan membuat web app sederhana menggunakan streamlit. Untuk melengkapi pengetahuan Anda tentang streamlit, kita akan mengenal berbagai basic element yang terdapat dalam streamlit pada materi kali ini.

Sebagai sebuah web app framework yang andal, streamlit telah menyediakan banyak pilihan element, widget, layout serta container untuk memastikan kita dapat membuat web app atau dashboard yang menarik dan interaktif. Nah, pada materi ini, kita hanya akan fokus pada bagian basic element dalam streamlit yang terdiri dari write, text, data display, dan chat. Yuk, kita bahas satu per satu!

Write

Basic element pertama yang akan kita bahas ialah write. Ia merupakan elemen streamlit yang digunakan untuk menampilkan sebuah output. Untuk menggunakan element ini, kita hanya perlu memanggil function `write()` dan diikuti inputan yang ingin ditampilkan.

Pada contoh pembuatan “Hello, world!” app, kita telah menggunakan function `write()` untuk menampilkan output dari argument markdown. Sebenarnya function ini dapat digunakan untuk menampilkan hal lain, seperti DataFrame, visualisasi data, dll. Berikut merupakan contoh penggunaannya untuk menampilkan DataFrame.

```
1. import streamlit as st
2. import pandas as pd
3.
4. st.write(pd.DataFrame({
5.     'c1': [1, 2, 3, 4],
6.     'c2': [10, 20, 30, 40],
7. }))
```

Kode di atas akan menghasilkan tampilan web app seperti berikut.

	c1	c2
0	1	10
1	2	20
2	3	30
3	4	40

Untuk melihat contoh lain dalam penggunaan function `write()`, Anda dapat mengunjungi dokumentasi berikut: [st.write\(\)](#).

Text

Elemen lain yang ada dalam streamlit ialah text (dokumentasi: [text](#)). Sesuai dengan namanya, ia merupakan elemen yang digunakan untuk menampilkan sebuah output

berupa text. Elemen ini memiliki banyak function yang bisa digunakan sesuai kebutuhan. Berikut merupakan beberapa pilihan yang tersedia saat ini.

- **markdown()**

Function ini digunakan untuk menampilkan output dari argument markdown. Berikut merupakan contoh kode untuk menggunakannya.

```
1. import streamlit as st
2.
3. st.markdown(
4.     """
5.     # My first app
6.     Hello, para calon praktisi data masa depan!
7.     """
8. )
```

- **title()**

Ini merupakan function yang digunakan untuk menampilkan teks dalam format judul. Kode yang dapat Anda gunakan untuk menerapkan function tersebut adalah seperti di bawah ini.

```
1. import streamlit as st
2.
3. st.title('Belajar Analisis Data')
```

- **header()**

Function ini digunakan untuk menampilkan output teks sebagai format header. Berikut contoh penulisan kode untuk menggunakannya.

```
1. import streamlit as st
2.
3. st.header('Pengembangan Dashboard')
```

- **subheader()**

Ini merupakan function yang digunakan untuk menampilkan output teks sebagai format subheader.

```
1. import streamlit as st
2.
```

3. `st.subheader('Pengembangan Dashboard')`

- **caption()**

Function berikutnya ialah `caption()`. Ia merupakan function yang digunakan untuk menampilkan output teks dalam ukuran kecil. Function ini biasanya digunakan untuk menampilkan caption, footnotes, dll. Contoh penggunaannya seperti di bawah ini.

```
1. import streamlit as st
2.
3. st.caption('Copyright (c) 2023')
```

- **code()**

Pada beberapa case, kita harus menampilkan potongan kode ke dalam streamlit app (web app yang dibuat menggunakan streamlit). Untuk menjawab hal ini, streamlit telah menyediakan sebuah function bernama `code()`. Kode di bawah ini merupakan contoh penggunaan dari function tersebut.

```
1. import streamlit as st
2.
3. code = """def hello():
4.     print("Hello, Streamlit!")"""
5. st.code(code, language='python')
```

Kode di atas akan menghasilkan tampilan seperti berikut.

```
def hello():
    print("Hello, Streamlit!")
```

- **text()**

Function selanjutnya ialah `text()`. Function ini digunakan untuk menampilkan sebuah normal teks. Berikut merupakan contoh kode untuk mengguankannya.

```
1. import streamlit as st
```

```
2.  
3. st.text('Halo, calon praktisi data masa depan.')
```

- **latex()**

Function terakhir yang dapat digunakan untuk menampilkan elemen teks ialah `latex()`. Sesuai namanya, function tersebut digunakan untuk menampilkan *mathematical expression* yang ditulis dalam [format LaTeX](#). Berikut contoh kode untuk menggunakan function `latex()`.

```
1. import streamlit as st  
2.  
3. st.latex(r"""  
4.     \sum_{k=0}^{n-1} ar^k =  
5.     a \left(\frac{1-r^n}{1-r}\right)  
6. """)
```

Kode tersebut akan menghasilkan tampilan mathematical expression seperti berikut.

$$\sum_{k=0}^{n-1} ar^k = a \left(\frac{1 - r^n}{1 - r} \right)$$

Data Display

Basic element selanjutnya yang akan kita bahas ialah data display (dokumentasi: [data display](#)). Ia merupakan elemen yang digunakan untuk menampilkan data secara cepat dan interaktif ke dalam streamlit app yang kita buat. Elemen ini memiliki beberapa function seperti berikut.

- **dataframe()**

Function pertama yang bisa kita gunakan untuk menampilkan data ke dalam streamlit app ialah `dataframe()`. Ia merupakan function yang digunakan untuk menampilkan DataFrame sebagai sebuah tabel interaktif. Pada function ini, kita bisa mengatur ukuran dari table yang ingin ditampilkan menggunakan

parameter `width` dan `height`. Berikut merupakan contoh kode untuk menampilkan data menggunakan function `dataframe()`.

```
1. import pandas as pd
2. import streamlit as st
3.
4. df = pd.DataFrame({
5.     'c1': [1, 2, 3, 4],
6.     'c2': [10, 20, 30, 40],
7. })
8.
9. st.dataframe(data=df, width=500, height=150)
```

- **table()**

Selain `dataframe()`, kita juga bisa menggunakan function `table()` untuk menampilkan data ke dalam streamlit app. Ia dapat digunakan untuk menampilkan data dalam bentuk static table. Berikut merupakan contoh penggunaannya.

```
1. import pandas as pd
2. import streamlit as st
3.
4. df = pd.DataFrame({
5.     'c1': [1, 2, 3, 4],
6.     'c2': [10, 20, 30, 40],
7. })
8. st.table(data=df)
```

- **metric()**

Ketika membuat dashboard, terkadang kita perlu menampilkan sebuah metrik tertentu. Untuk melakukan hal ini, kita bisa memanfaatkan function `metric()`. Function ini dapat membantu kita untuk menampilkan sebuah metrik tertentu beserta detailnya seperti label, value serta besar perubahan nilainya. Berikut merupakan contoh kode untuk menggunakannya.

```
1. import streamlit as st
```

```

2.
3. st.metric(label="Temperature", value="28 °C", delta="1.2 °C")

```

Kode di atas akan menghasilkan tampilan berikut.

Temperature
 28 °C
 ↑ 1.2 °C

- **json()**

Selain bentuk DataFrame atau tabel, terkadang kita juga perlu menampilkan data dalam format JSON. Streamlit telah menyediakan function `json()` untuk menampilkan data dalam format JSON. Berikut merupakan contoh penggunaannya.

```

1. import streamlit as st
2.
3. st.json({
4.     'c1': [1, 2, 3, 4],
5.     'c2': [10, 20, 30, 40],
6. })

```

Kode di atas akan menghasilkan tampilan berikut.

```

{
  "c1" : [
    0 : 1
    1 : 2
    2 : 3
    3 : 4
  ]
  "c2" : [
    0 : 10
    1 : 20
    2 : 30
    3 : 40
  ]
}

```

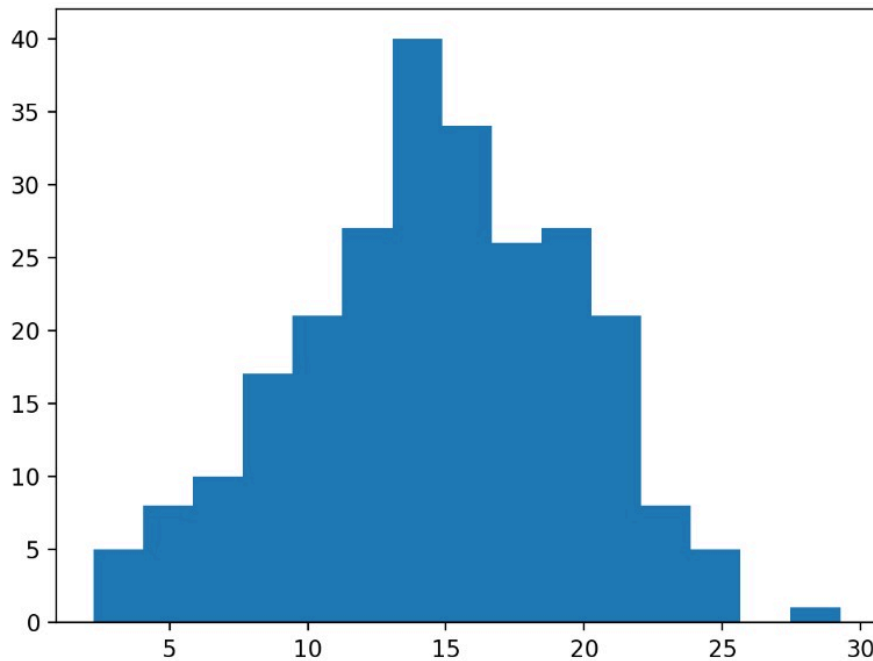
Chart

Basic element terakhir yang perlu kita ketahui ialah chart. Sesuai namanya, elemen ini dapat digunakan untuk menampilkan grafik visualisasi data ke dalam streamlit app. Elemen inilah yang akan sering kita gunakan untuk membuat dashboard nantinya. Sebenarnya streamlit telah menyediakan banyak sekali function untuk mendukung berbagai library visualisasi data (dokumentasi: [chart](#)). Namun, pada materi ini, kita hanya akan fokus pada function `pyplot()`.

Function `pyplot()` dapat digunakan untuk menampilkan grafik visualisasi data yang dibuat menggunakan matplotlib. Berikut merupakan contoh kode untuk menggunakannya.

```
1. import matplotlib.pyplot as plt
2. import numpy as np
3. import streamlit as st
4.
5. x = np.random.normal(15, 5, 250)
6.
7. fig, ax = plt.subplots()
8. ax.hist(x=x, bins=15)
9. st.pyplot(fig)
```

Kode di atas akan menampilkan grafik visualisasi data berikut di dalam streamlit app.



Oke, itulah berbagai basic element yang ada dalam streamlit. Pemahaman terhadap basic element ini sangat penting untuk mendukung Anda dalam membuat sebuah dashboard nantinya.

Nah, untuk melengkapi pengetahuan Anda, kita akan membahas basic widgets dalam streamlit pada materi selanjutnya. *So*, tanpa berlama-lama lagi. Yuk, kita lanjut!

BAB 3 Basic Widgets dalam Streamlit

Yey, pada materi sebelumnya, kita telah mengenal berbagai basic element yang ada dalam streamlit. Hal tersebut tentunya akan sangat membantu kita dalam membuat dashboard. Namun, untuk menghasilkan sebuah dashboard yang interaktif, kita memerlukan komponen lain seperti *widget* (elemen *Graphical User Interface* yang memungkinkan pengguna untuk berinteraksi dengan aplikasi).

Nah, pada materi ini, kita akan berkenalan dengan berbagai basic widget yang telah disediakan oleh streamlit. Untuk membantu kita menghasilkan web app yang interaktif, streamlit telah menyediakan banyak sekali pilihan widget yang bisa

disesuaikan dengan kebutuhan. Agar lebih mudah dalam memahami seluruh widget tersebut, pada materi ini kita akan membaginya ke dalam dua kategori, yaitu *input widget* dan *button widget*.

Input Widget

Kategori widget yang akan kita bahas ialah input widget. Ia merupakan kategori widget yang memungkinkan pengguna untuk memberikan input ke dalam streamlit app. Terdapat beberapa widget yang termasuk kategori ini, seperti *text input*, *number input*, *date input*, dll. Yuk, kita bahas satu per satu!

Text input

Text input merupakan widget yang digunakan untuk memperoleh inputan berupa *single-line text*. Kita bisa menggunakan function `text_input()` untuk membuat widget ini. Berikut merupakan contoh kode untuk membuatnya.

```
1. import streamlit as st
2.
3. name = st.text_input(label='Nama lengkap', value='')
4. st.write('Nama: ', name)
```

Kode di atas akan menghasilkan sebuah widget seperti berikut.

Nama lengkap

Fajri

Nama: Fajri

Text-area

Text area merupakan widget yang memungkinkan pengguna untuk menginput *multi-line text*. Untuk membuat widget ini, streamlit telah menyediakan function bernama `text_area()`. Kode di bawah ini merupakan contoh kode untuk menggunakan function tersebut.

```
1. import streamlit as st
2.
3. text = st.text_area('Feedback')
4. st.write('Feedback: ', text)
```

Berikut merupakan tampilan widget yang diperoleh dari kode di atas.

Feedback

Nice! Bintang 5 :)|

Feedback: Nice! Bintang 5 :)

Number input

Widget berikutnya yang akan kita bahas ialah number input . Ia merupakan widget yang digunakan untuk memperoleh inputan berupa angka dari pengguna. Anda dapat menggunakan contoh kode berikut untuk membuat number input widget menggunakan function `number_input()`.

```
1. import streamlit as st
2.
3. number = st.number_input(label='Umur')
4. st.write('Umur: ', int(number), ' tahun')
```

Gambar di bawah ini merupakan tampilan widget yang dihasilkan dari contoh kode tersebut.

Umur

23,00 - +

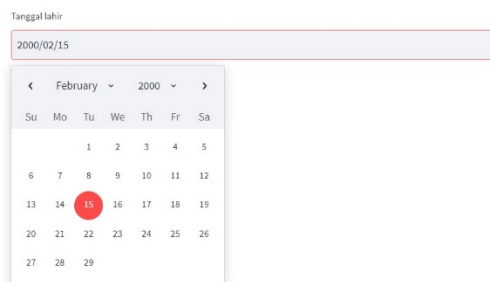
Umur: 23 tahun

Date input

Selain inputan berupa angka dan teks, pada beberapa kasus kita juga membutuhkan input berupa tanggal dari pengguna melalui date input widget. Kita dapat membuat widget tersebut menggunakan function `date_input()`. Berikut merupakan contoh kode untuk menggunakannya.

1. `import datetime`
2. `import streamlit as st`
- 3.
4. `date = st.date_input(label='Tanggal lahir', min_value=datetime.date(1900, 1, 1))`
5. `st.write('Tanggal lahir:', date)`

Kode tersebut akan menghasilkan tampilan widget seperti berikut.



File uploader

Widget selanjutnya yang akan kita bahas ialah file uploader. Widget ini memungkinkan kita meminta pengguna untuk meng-*upload* sebuah berkas tertentu ke dalam web app. Kita dapat membuat file uploader widget menggunakan function `file_uploader()` seperti pada contoh kode berikut.

1. `import streamlit as st`
2. `import pandas as pd`
- 3.
4. `uploaded_file = st.file_uploader('Choose a CSV file')`
- 5.
6. `if uploaded_file:`
7. `df = pd.read_csv(uploaded_file)`
8. `st.dataframe(df)`

Kode di atas akan menghasilkan widget seperti berikut.

Choose a CSV file

Drag and drop file here
Limit 200MB per file

Browse files

Telco-Customer-Churn.csv 438.5KB

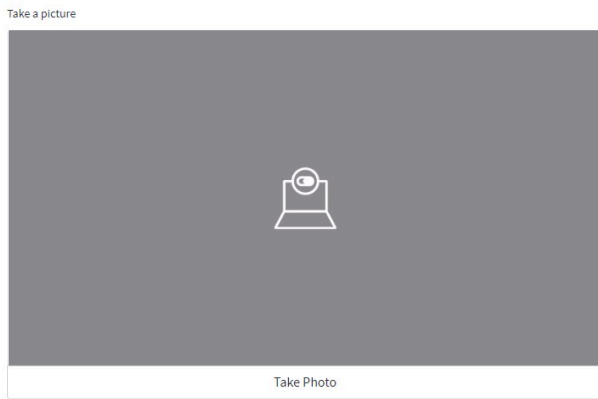
	customerID	gender	SeniorCitizen	Partner	StreamingTV	PhoneService	InternetService	Paperless
0	7590-VHVEG	Female	No	Yes	No	No	DSL	Yes
1	5575-GNVDE	Male	No	No	No	Yes	DSL	No
2	3668-QPYBK	Male	No	No	No	Yes	DSL	Yes
3	7795-CFOCW	Male	No	No	No	No	DSL	No
4	9237-HQITU	Female	No	No	No	Yes	Fiber optic	Yes
5	9305-CDSKC	Female	No	No	Yes	Yes	Fiber optic	Yes
6	1452-KIOVK	Male	No	No	Yes	Yes	Fiber optic	Yes
7	6713-OKOMC	Female	No	No	No	No	DSL	No
8	7892-POOKP	Female	No	Yes	Yes	Yes	Fiber optic	Yes
9	6388-TABGU	Male	No	No	No	Yes	DSL	No

Camera input

Selain file uploader, streamlit juga menyediakan camera input widget yang dapat digunakan untuk meminta user mengambil gambar melalui *webcam* sekaligus mengunggahnya. Hal ini tentunya dilakukan dengan persetujuan pengguna. Berikut merupakan contoh kode untuk membuat camera input widget menggunakan function `camera_input()`.

1. `import streamlit as st`
2. `picture = st.camera_input('Take a picture')`
3. `if picture:`
4. `st.image(picture)`

Berikut merupakan tampilan widget yang akan diperoleh ketika menjalankan kode di atas.



Button Widgets

Oke, kategori widget selanjutnya yang akan kita bahas ialah button widget. Ia merupakan kategori widget yang terdiri dari button, checkbox, radio button, dll.

Button

Button merupakan widget untuk menampilkan tombol interaktif. Tombol tersebut dapat digunakan pengguna untuk melakukan aksi tertentu. Untuk membuat widget ini, kita bisa menggunakan function `button()` seperti contoh berikut.

1. `import streamlit as st`
- 2.
3. `if st.button('Say hello'):`
4. `st.write('Hello there')`

Berikut merupakan button widget yang dihasilkan dari kode di atas.

Say hello

Hello there

Checkbox

Checkbox merupakan widget yang digunakan untuk menampilkan sebuah *checkboxlist* untuk pengguna. Kita bisa menggunakan function `checkbox()` untuk membuat dan menampilkan checklist dalam streamlit app. Berikut contoh kode untuk melakukannya.

```
1. import streamlit as st
2.
3. agree = st.checkbox('I agree')
4.
5. if agree:
6.     st.write('Welcome to MyApp')
```

Kode di atas akan menghasilkan tampilan berikut.

☒ I agree

Welcome to MyApp

Radio button

Selain button dan checkbox, terkadang kita juga membutuhkan radio button untuk menghasilkan web app yang interaktif. Ia merupakan widget yang memungkinkan pengguna untuk memilih satu dari beberapa pilihan yang ada. Untuk membuat widget ini, kita bisa menggunakan function `radio()` seperti pada contoh kode berikut.

```
1. import streamlit as st
2.
3. genre = st.radio(
4.     label="What's your favorite movie genre",
```

5. `options=('Comedy', 'Drama', 'Documentary'),`
6. `horizontal=False`
7. `)`

Berikut merupakan tampilan widget yang dihasilkan dari kode di atas.

What's your favorite movie genre

- ☒ Comedy
- ☐ Drama
- ☐ Documentary

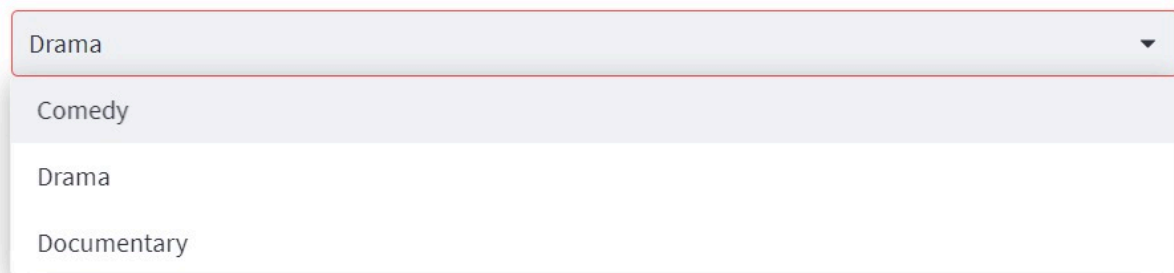
Select Box

Select box merupakan widget yang memungkinkan pengguna untuk memilih salah satu dari beberapa pilihan yang ada. Ia merupakan opsi alternatif dari radio button. Streamlit telah menyediakan function `selectbox()` untuk membuat select box widget. Berikut contoh penggunaannya.

1. `import streamlit as st`
- 2.
3. `genre = st.selectbox(`
4. `label="What's your favorite movie genre",`
5. `options=('Comedy', 'Drama', 'Documentary')`
6. `)`

Berikut merupakan tampilan select box dari kode di atas.

What's your favorite movie genre



A single-select dropdown menu with a light gray background and a red border. The selected option is 'Drama'. The dropdown is open, showing a list of options: 'Comedy', 'Drama', and 'Documentary'.

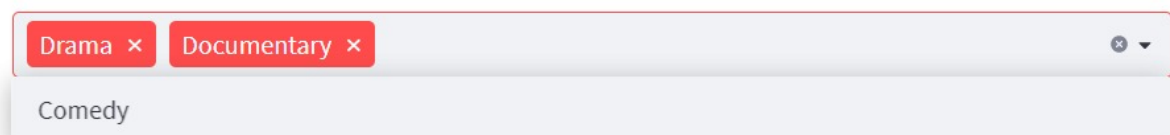
Multiselect

Widget lain yang harus kita ketahui ialah multiselect. Ia merupakan widget yang digunakan agar user dapat memilih lebih dari satu pilihan dari sekumpulan pilihan yang ada. Untuk mempermudah kita dalam membuat multiselect widget, streamlit telah menyediakan function bernama `multiselect()`. Berikut contoh kode untuk menggunakan function tersebut.

```
1. import streamlit as st
2.
3. genre = st.multiselect(
4.     label="What's your favorite movie genre",
5.     options=('Comedy', 'Drama', 'Documentary')
6. )
```

Kode di atas akan menghasilkan tampilan multiselect widget seperti berikut.

What's your favorite movie genre



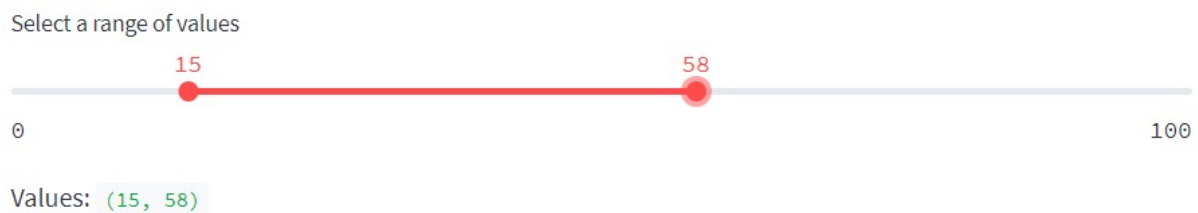
A multiselect widget with a light gray background and a red border. The selected options are 'Drama' and 'Documentary', each with a red 'x' button to remove it. The dropdown is open, showing a list of options: 'Comedy'.

Slider

Slider merupakan widget yang memungkinkan pengguna untuk memilih nilai (atau range nilai) dari sebuah range nilai yang telah ditentukan. Streamlit telah menyediakan function `slider()` untuk membuat slider widget. Berikut merupakan contoh penggunaannya.

```
1. values = st.slider(  
2.     label='Select a range of values',  
3.     min_value=0, max_value=100, value=(0, 100))  
4. st.write('Values:', values)
```

Kode di atas akan menghasilkan widget seperti berikut.



Nah, itulah beberapa pilihan widget yang disediakan oleh streamlit. Anda dapat melihat lebih banyak pilihan widget pada dokumentasi berikut: [widgets in streamlit](#). Pengetahuan terkait basic widget ini akan sangat membantu Anda dalam membuat dashboard yang interaktif nantinya.

Oke, sebelum kita belajar membuat dashboard, terdapat satu materi lagi yang harus dipahami, yaitu terkait layout dalam streamlit. *So*, tetap semangat, ya!

BAB 4 Basic Layouts dalam Streamlit

Sejauh ini Anda telah mengenal berbagai hal mulai dari basic element hingga beberapa pilihan widget yang ada dalam streamlit. Semua hal tersebut tentunya akan sangat membantu kita dalam membuat sebuah dashboard yang interaktif.

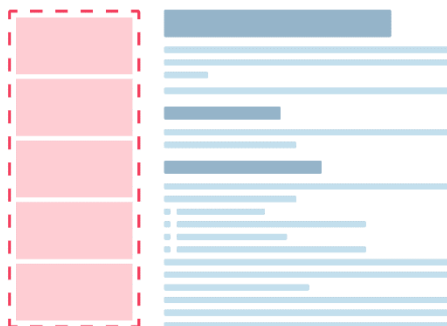
Apakah itu sudah cukup? Tentu saja belum!

Untuk membuat dashboard yang rapi, kita perlu belajar cara mengatur layout atau tampilan dari sebuah dashboard. Nah, pada materi kali ini, kita akan mengupas tuntas tentang basic layout dalam streamlit.

Sebagai sebuah web app framework, streamlit telah menyediakan berbagai pilihan layout yang dapat digunakan untuk mengatur tampilan web app (atau dashboard) yang akan dibuat. Pilihan layout yang tersedia antara lain *sidebar*, *columns*, *tabs*, *expander*, serta *container*. Tentunya setiap pilihan layout tersebut memiliki fungsi dan kegunaannya masing-masing. Yuk, kita bahas satu per satu!

Sidebar

Elemen layout pertama yang akan kita bahas ialah sidebar. Ia merupakan area yang berada di samping konten utama. Pada streamlit, posisi sidebar secara default berada di bagian kiri dari konten utama dan dapat memuat berbagai hal mulai dari gambar, teks, hingga widget.



Untuk menambahkan sebuah elemen atau widget ke dalam sidebar, kita bisa menggunakan notasi `with` yang diikuti sebuah object bernama `sidebar` yang telah disediakan oleh streamlit. Berikut merupakan contoh kode untuk menambah sebuah elemen dan widget ke dalam sidebar.

1. `import streamlit as st`

```

2.
3. st.title('Belajar Analisis Data')
4.
5. with st.sidebar:
6.
7.     st.text('Ini merupakan sidebar')
8.
9.     values = st.slider(
10.         label='Select a range of values',
11.         min_value=0, max_value=100, value=(0, 100)
12.     )
13.     st.write('Values:', values)

```

Kode di atas akan menghasilkan tampilan streamlit app seperti berikut.

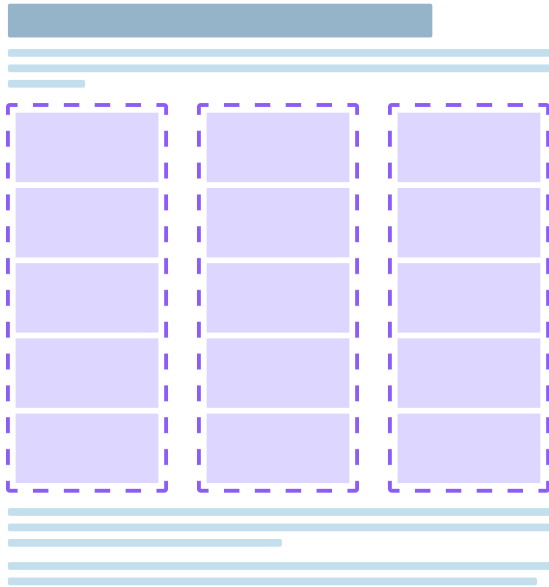


Belajar Analisis Data

Bagaimana cukup mudah bukan untuk membuat sidebar di streamlit? (dokumentasi: [sidebar](#)). Nah, sidebar ini dapat kita gunakan untuk menampung gambar logo serta widget yang digunakan sebagai filter. Namun, hal ini akan kita bahas pada materi berikutnya.

Columns

Columns merupakan elemen layout yang memungkinkan kita untuk mengatur tampilan pada konten utama ke dalam beberapa kolom sesuai kebutuhan. Gambar berikut merupakan ilustrasi tampilan dari elemen layout ini.



Untuk membuat column dalam streamlit app, kita harus membuat object dari setiap kolom terlebih dahulu. Hal ini dapat dilakukan dengan memanfaatkan function `columns()`. Selanjutnya, kita hanya perlu menambahkan sebuah elemen atau widget ke dalam column tersebut menggunakan notasi `with`. Berikut merupakan contoh kode untuk membuat column dalam streamlit.

1. `import streamlit as st`
- 2.
3. `st.title('Belajar Analisis Data')`
4. `col1, col2, col3 = st.columns(3)`
- 5.
6. `with col1:`


```
7. st.header("Kolom 1")
8. st.image("https://static.streamlit.io/examples/cat.jpg")
9.
10. with col2:
11.     st.header("Kolom 2")
12.     st.image("https://static.streamlit.io/examples/dog.jpg")
13.
14. with col3:
15.     st.header("Kolom 3")
16.     st.image("https://static.streamlit.io/examples/owl.jpg")
```

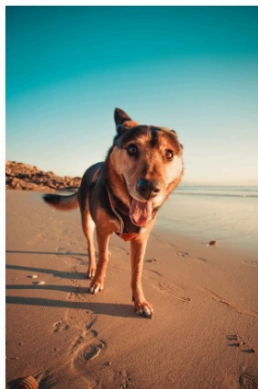
Berikut tampilan streamlit app yang dihasilkan dari kode di atas.

Belajar Analisis Data

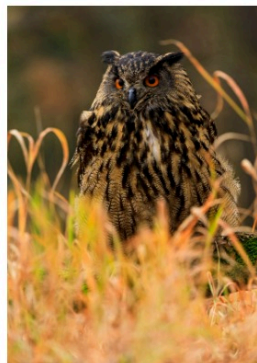
Kolom 1



Kolom 2



Kolom 3



Sebenarnya, kita bisa dengan bebas mengatur ukuran dari column yang dibuat. Nah, untuk melakukan hal ini, kita harus memasukkan sebuah *list* yang berisi

perbandingan ukuran dari kolom yang akan dibuat. Sebagai contoh, kode di bawah ini akan menampilkan 3 buah kolom dengan perbandingan **2:1:1**.

```
1. import streamlit as st
2.
3. st.title('Belajar Analisis Data')
4. col1, col2, col3 = st.columns([2, 1, 1])
5.
6. with col1:
7.     st.header("Kolom 1")
8.     st.image("https://static.streamlit.io/examples/cat.jpg")
9.
10. with col2:
11.     st.header("Kolom 2")
12.     st.image("https://static.streamlit.io/examples/dog.jpg")
13.
14. with col3:
15.     st.header("Kolom 3")
16.     st.image("https://static.streamlit.io/examples/owl.jpg")
```

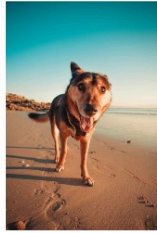
Berikut merupakan tampilan yang akan dihasilkan dari kode tersebut.

Belajar Analisis Data

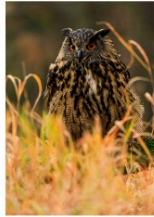
Kolom 1



Kolom 2

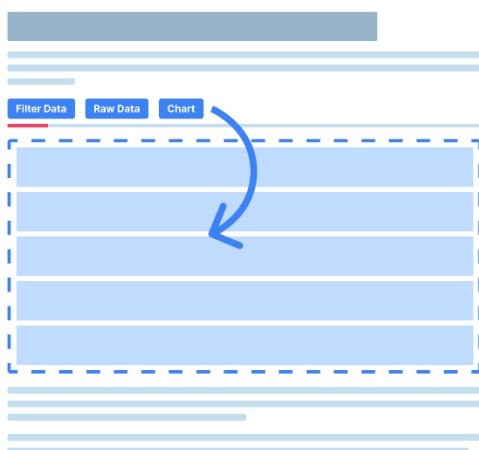


Kolom 3



Tabs

Elemen layout berikutnya yang terdapat dalam streamlit ialah tabs. Ia merupakan elemen layout yang memungkinkan kita untuk menambahkan beberapa tabs ke dalam konten utama. Hal ini tentunya akan sangat membantu kita dalam menghasilkan dashboard yang interaktif.



Sama halnya dengan columns yang sebelumnya kita bahas, untuk membuat tabs, kita harus membuat object dari setiap tab menggunakan function `tabs()` yang diikuti

dengan list dari nama dari setiap tab. Berikut contoh kode untuk membuat tabs dalam streamlit app.

```
1. import streamlit as st
2.
3. st.title('Belajar Analisis Data')
4. tab1, tab2, tab3 = st.tabs(["Tab 1", "Tab 2", "Tab 3"])
5.
6. with tab1:
7.     st.header("Tab 1")
8.     st.image("https://static.streamlit.io/examples/cat.jpg")
9.
10. with tab2:
11.     st.header("Tab 2")
12.     st.image("https://static.streamlit.io/examples/dog.jpg")
13.
14. with tab3:
15.     st.header("Tab 3")
16.     st.image("https://static.streamlit.io/examples/owl.jpg")
```

Berikut merupakan tampilan tabs yang akan Anda peroleh dari kode di atas.

Belajar Analisis Data

Tab 1 Tab 2 **Tab 3**

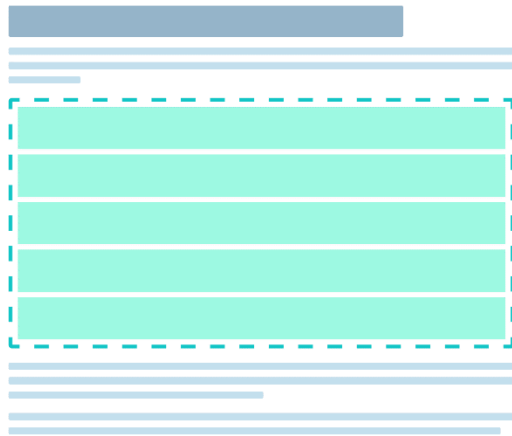
Tab 3



Nah, itulah cara membuat tabs dalam streamlit app (dokumentasi: [tabs](#)). Cukup mudah, bukan?

Container

Container merupakan elemen layout dalam streamlit yang memungkinkan kita untuk membuat sebuah wadah untuk menampung suatu atau beberapa elemen dengan ukuran yang tetap. Container ini dapat kita gunakan untuk menghasilkan dashboard yang rapi.

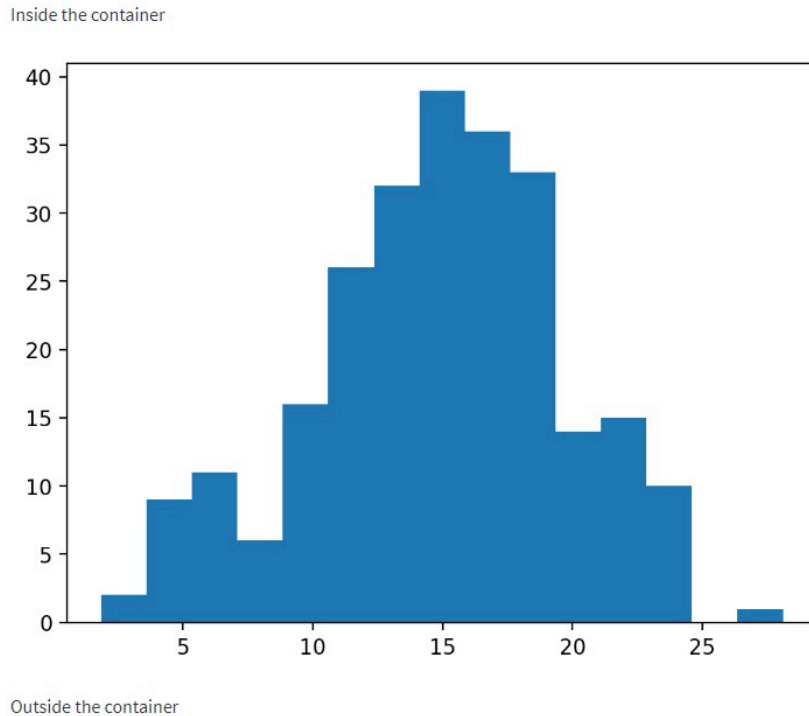


Untuk membuat container, kita bisa menggunakan notasi `with` dan diikuti function `container()`. Kode di bawah ini merupakan contoh kode untuk membuat container dalam streamlit app.

1. `import streamlit as st`
2. `import matplotlib.pyplot as plt`
3. `import numpy as np`
- 4.
5. `with st.container():`
6. `st.write("Inside the container")`
- 7.
8. `x = np.random.normal(15, 5, 250)`
- 9.
10. `fig, ax = plt.subplots()`
11. `ax.hist(x=x, bins=15)`
12. `st.pyplot(fig)`
- 13.

```
14. st.write("Outside the container")
```

Berikut merupakan tampilan streamlit app dari kode di atas.



Ketika melihat gambar di atas, mungkin Anda tidak melihat adanya perbedaan dari tampilan sebelum dan sesudah menggunakan container. Hal ini terjadi karena kita masih belum memiliki banyak elemen yang ingin ditampilkan.

Expander

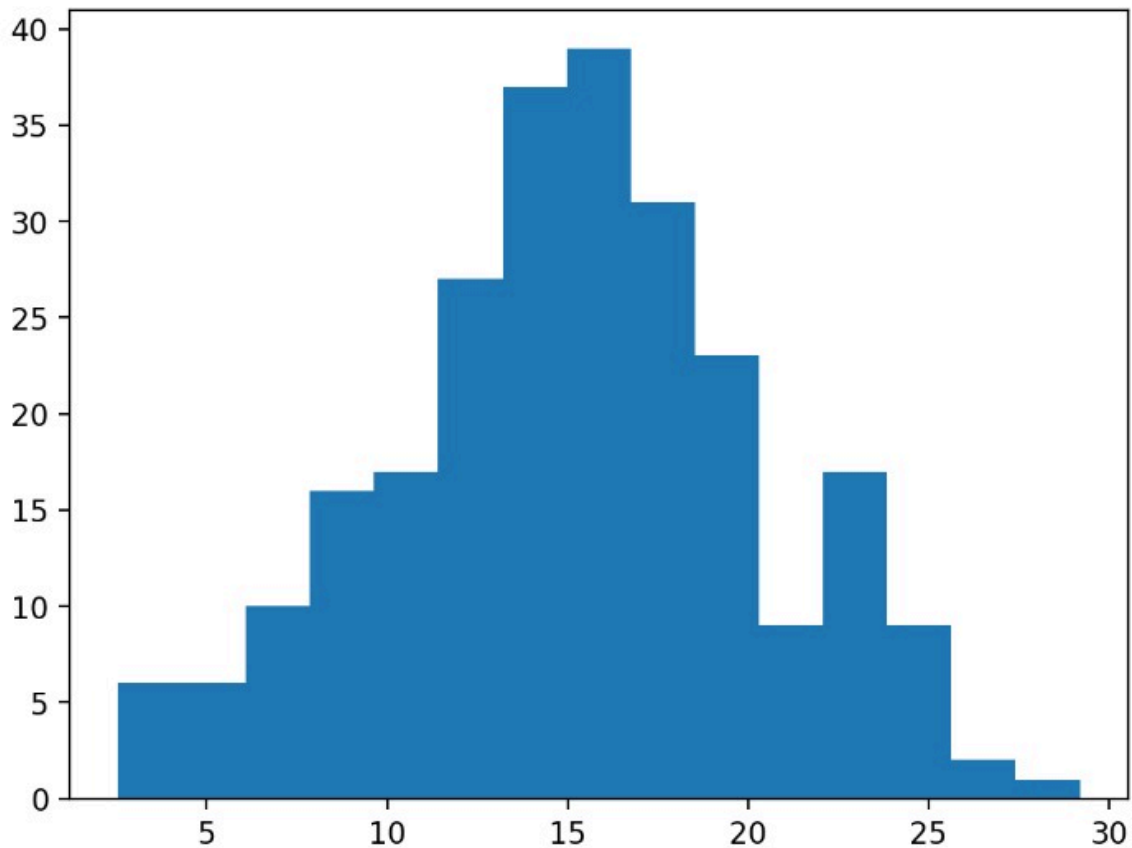
Elemen layout selanjutnya yang tidak kalah penting ialah expander. Anda dapat menganggap elemen layout ini sebagai sebuah container yang dapat diperluas atau dicituk secara interaktif.



Untuk membuat elemen layout ini, kita bisa menggunakan notasi `with` dan diikuti dengan function `expander()` seperti pada contoh kode berikut.

1. `import streamlit as st`
2. `import matplotlib.pyplot as plt`
3. `import numpy as np`
- 4.
5. `x = np.random.normal(15, 5, 250)`
- 6.
7. `fig, ax = plt.subplots()`
8. `ax.hist(x=x, bins=15)`
9. `st.pyplot(fig)`
- 10.
11. `with st.expander("See explanation"):`
12. `st.write("Hello World")`

Berikut merupakan tampilan streamlit app dari kode di atas.



See explanation



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Seperti yang terdapat pada gambar di atas, kita bisa menggunakan expander untuk menampung penjelasan atau keterangan lanjutan dari sebuah visualisasi data yang ditampilkan dalam sebuah dashboard.

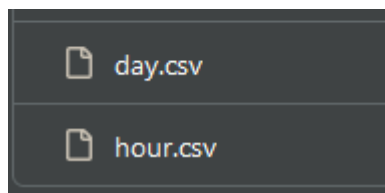
Nah, itulah beberapa pilihan layout yang disediakan oleh streamlit untuk membantu kita dalam membuat web app yang rapi dan interaktif. Apabila ingin mempelajari topik ini lebih lanjut, Anda dapat mengunjungi dokumentasi berikut: [layouts and containers](#).

BAB 5 Proyek Dicoding

Persiapan Google Colab

Pertama-tama sebelum memasuki StreamLit mari kita proses dulu datanya melalui Google Colab ataupun Jupyter Notebook.

1. Mengambil Dataset yang digunakan. Disini kita akan menggunakan Bike Sharing Dataset yang disediakan oleh Hadi Fanaee-T.



2. Setelah itu mari kita buat pertanyaan bisnis yang ingin diselesaikan melalui visualisasi Data nya.

1. musim apa yang memiliki sewa tertinggi per harinya?
2. Bagaimana pola penyewaan sepeda berdasarkan bulan dan jam?
3. adakah pengaruh cuaca terhadap sewa sepeda per harinya?
4. perbandingan jumlah sewa setiap hari, mana hari yang memiliki sewa tertinggi dan terendah?

3. Lalu saatnya melakukan pemrograman di dalam Google Colab.
 - a. Import Library, dan melakukan Data Wrangling

```
#import library yang dibutuhkan  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns|
```

```
#mengimport dataset hour  
jam = pd.read_csv("/kaggle/input/bike-sharing-dataset/hour.csv")  
jam.head()
```

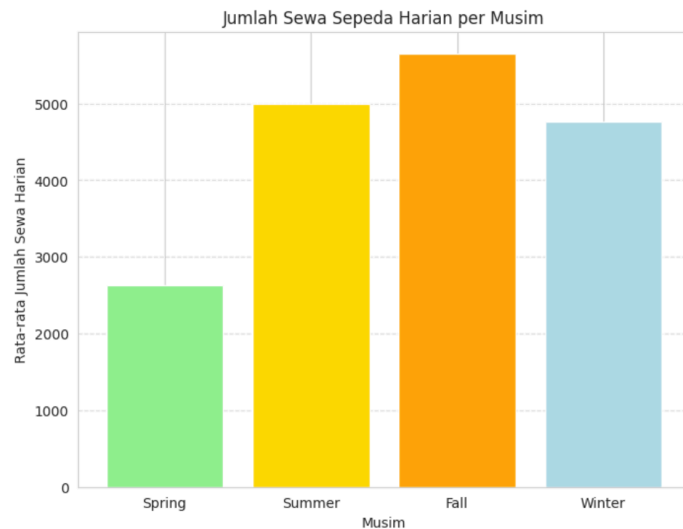
```
#mengimport dataset day
```

```
hari = pd.read_csv("/kaggle/input/bike-sharing-dataset/day.csv")  
hari.head()
```

b. Menjawab pertanyaan bisnisnya.

1. musim apa yang memiliki sewa tertinggi per harinya dan per jamnya?

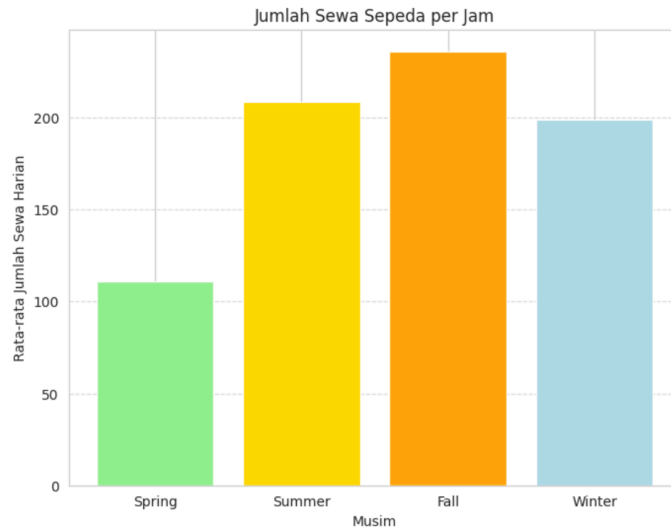
```
1: seasonal_data = df.groupby('season_daily')['cnt_daily'].mean()  
season_names = ['Spring', 'Summer', 'Fall', 'Winter']  
  
plt.figure(figsize=(8, 6))  
plt.bar(season_names, seasonal_data, color=['lightgreen', 'gold', 'orange', 'lightblue'])  
plt.xlabel('Musim')  
plt.ylabel('Rata-rata Jumlah Sewa Harian')  
plt.title('Jumlah Sewa Sepeda Harian per Musim')  
plt.grid(axis='y', linestyle='--', alpha=0.7)  
plt.show()
```



dari grafik tersebut bisa disimpulkan bahwa jumlah sewa sepeda lebih banyak pada Fall dengan sewa per hari bisa mendapatkan sewa sampe 5000+, tempat kedua diikuti oleh summer dengan hasil 5000 penyewa, tempat ketiga adalah winter dengan --4700 penyewa dan terakhir adalah spring dengan --2500 penyewa.

```
seasonal_data = df.groupby('season_hourly')['cnt_hourly'].mean()
season_names = ['Spring', 'Summer', 'Fall', 'Winter']

plt.figure(figsize=(8, 6))
plt.bar(season_names, seasonal_data, color=['lightgreen', 'gold', 'orange', 'lightblue'])
plt.xlabel('Musim')
plt.ylabel('Rata-rata Jumlah Sewa Harian')
plt.title('Jumlah Sewa Sepeda per Jam')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

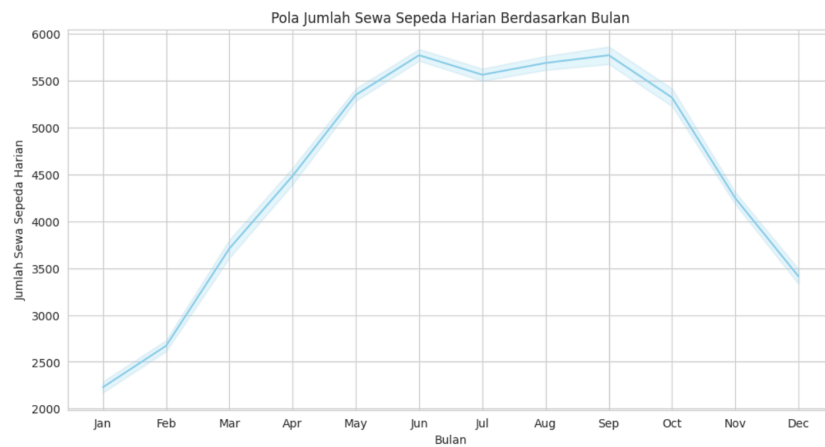


[+ Code](#) [+ Markdown](#)

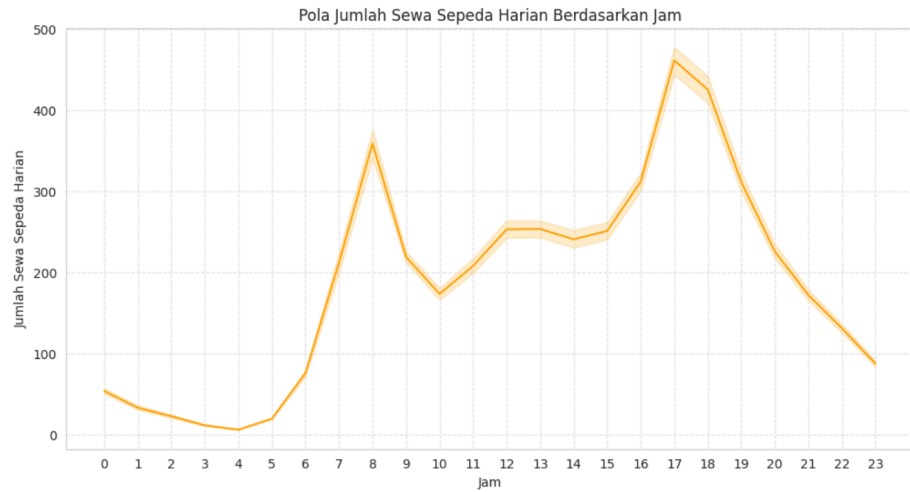
dari grafik tersebut bisa disimpulkan bahwa jumlah sewa sepeda per jam lebih banyak pada Fall dengan sewa per jam bisa mendapatkan sewa sampai +250, tempat kedua diikuti oleh summer dengan hasil 210 penyewa, tempat ketiga adalah winter dengan 190 penyewa dan terakhir adalah spring dengan 110 penyewa.

2. Bagaimana pola penyewaan sepeda berdasarkan bulan, jam?

```
sns.set_style("whitegrid")
plt.figure(figsize=(12, 6))
sns.lineplot(x="mnth_daily", y="cnt_daily", data=df, color='skyblue')
plt.title("Pola Jumlah Sewa Sepeda Harian Berdasarkan Bulan")
plt.xlabel("Bulan")
plt.ylabel("Jumlah Sewa Sepeda Harian")
plt.xticks(range(1, 13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.show()
```



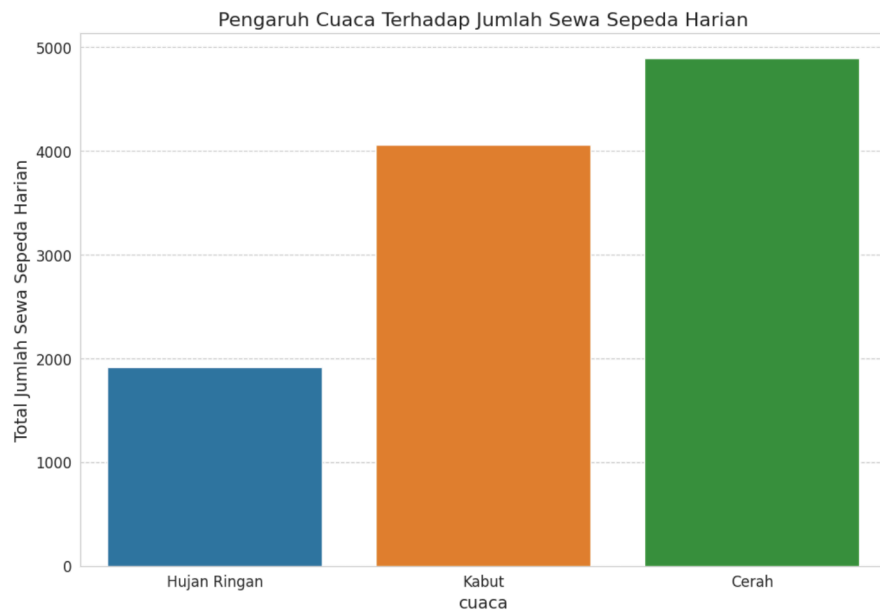
```
# Pola berdasarkan jam
sns.set_style("whitegrid")
plt.figure(figsize=(12, 6))
sns.lineplot(x="hr", y="cnt_hourly", data=df, color='orange')
plt.title("Pola Jumlah Sewa Sepeda Harian Berdasarkan Jam")
plt.xlabel("Jam")
plt.ylabel("Jumlah Sewa Sepeda Harian")
plt.xticks(range(24))
plt.grid(axis='both', linestyle='--', alpha=0.5)
plt.show()
```



kedua grafik diatas dapat dilihat bahwa terdapat pola yaitu penyewaan sepeda akan terus meningkat dari musim semi (spring) sampai musim panas dan akan menurun sampai akhir musim, kemudian pada musim gugur tingkat sewa sepeda akan naik sampai menyentuh level tertinggi di musim gugur. Jadi penyewaan sepeda lebih banyak terjadi pada bulan 6 dan bulan 9 dan menurun pada bulan 10. Sedangkan berdasarkan jam jumlah sewa sepeda akan meningkat sekitar jam 8 pagi dan sekitar jam 3 sore akan mengalami kenaikan sampe pukul 5 sore mungkin disaat orang mulai pulang dari segala aktivitasnya.

3. adakah pengaruh cuaca terhadap sewa sepeda per harinya??

```
# Pengaruh cuaca
avg_weather = df.groupby('weather_label')['cnt_daily'].mean().reset_index().sort_values("cnt_daily")
plt.figure(figsize=(12, 8))
sns.barplot(x="weather_label", y="cnt_daily", data=avg_weather, estimator=sum)
plt.title("Pengaruh Cuaca Terhadap Jumlah Sewa Sepeda Harian", fontsize=16)
plt.xlabel("cuaca", fontsize=14)
plt.ylabel("Total Jumlah Sewa Sepeda Harian", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.grid(axis='y', linestyle='--')
plt.show()
```



jumlah sewa sepeda meningkat ketika cuaca Cerah, Sedikit awan, Berawan sebagian, Berawan sebagian

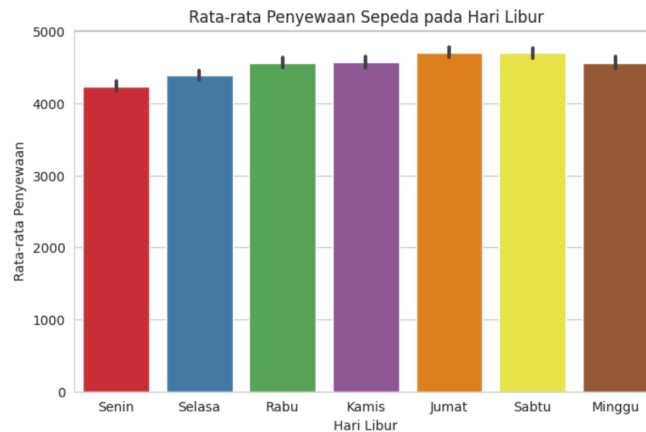
4. perbandingan jumlah sewa setiap hari, mana hari yang memiliki sewa tertinggi dan terendah?

```
# Perbandingan hari sewa sepeda setiap hari

plt.figure(figsize=(8, 5))
sns.barplot(x='weekday_daily', y='cnt_daily', data=df, palette='Set1')

plt.title('Rata-rata Penyewaan Sepeda pada Hari Libur')
plt.xlabel('Hari Libur')
plt.ylabel('Rata-rata Penyewaan')
plt.xticks([0, 1, 2, 3, 4, 5, 6], ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu'])

plt.show()
```



dari grafik diatas dapat dilihat bahwa jumlah sewa sepeda lebih banyak ketika hari jumat dan sabtu dan hari senin menjadi hari dengan sewa paling sedikit.

4. Setelah mengetahui bahwa datanya tidak perlu dilakukan cleaning, maka kita bisa menggunakannya ke dalam StreamLit.

Persiapan Visual Studio Code

Sebelum mulai mengerjakan latihan ini, terdapat beberapa hal yang harus disiapkan terlebih dahulu.

1. Instal berbagai library yang digunakan dengan beberapa perintah berikut.

```
1. pip install streamlit babel
```

2. Buka code editor favorit Anda dan buatlah sebuah proyek baru dengan nama **manabe baiku**.
3. Pindahkan berkas data yang sebelumnya sudah diunduh ke dalam folder proyek dashboard.

4. Arahkan Terminal/PowerShell/Command Prompt ke dalam path folder proyek dashboard.
5. Untuk menjalankan Dashboard bisa menggunakan perintah berikut.

```
1. streamlit run dashboard\dashboard.py
```

Menyiapkan DataFrame

Setelah tahap persiapan selesai, kini kita dapat mulai membuat dashboard dengan streamlit. Pada tahap awal tentunya kita perlu membuat sebuah berkas python dengan nama **dashboard.py**. Bukalah berkas tersebut dan tambahkan perintah berikut untuk mengimpor seluruh library yang dibutuhkan pada proyek ini.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import streamlit as st
```

Tahap berikutnya adalah menyiapkan data yang akan digunakan untuk membuat visualisasi data.

```
def create_sum_rental_df(df):
    sum_rental_df = df.reset_index()
    sum_rental_df.rename(columns={
        "cnt_hourly": "total_rental",
        "registered_hourly": "registered_user",
    }, inplace=True)
    return sum_rental_df

df = pd.read_csv("df_bike.csv")
```

Membuat Komponen Antarmuka Pengguna

Setelah berhasil menyiapkan DataFrame yang akan digunakan, tahap berikutnya ialah membuat antarmuka pada dashboard yang akan dibuat. Pada latihan ini, kita akan menggunakan widget date input sebagai filternya dan akan ditempatkan pada bagian sidebar. Selain itu, untuk alasan estetika, kita juga perlu menambahkan logo perusahaan ke dalam sidebar tersebut.

Berikut merupakan contoh kode yang dapat Anda gunakan untuk membuat estetika dengan widget date input serta menambahkan logo perusahaan pada sidebar.

```
# Filter data
df['dteday'] = pd.to_datetime(df['dteday'])
min_date = df["dteday"].min()
max_date = df["dteday"].max()

with st.sidebar:
    # Menambahkan logo perusahaan
    st.image("https://images.unsplash.com/photo-1507035895480-2b3156c31fc8?q=80&w=3540&auto=format&fit=crop&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D")

    # Mengambil start_date & end_date dari date_input
    start_date, end_date = st.date_input(
        label='Rentang Waktu',min_value=min_date,
        max_value=max_date,
        value=[min_date, max_date]
    )
```

Kode di atas akan menghasilkan `start_date` dan `end_date` yang selanjutnya akan digunakan untuk estetika. Berikut merupakan tampilan sidebar yang akan dihasilkan dari kode tersebut.



Melengkapi Dashboard dengan Berbagai Visualisasi Data

Oke, setelah membuat widget sidebar dan menyiapkan seluruh DataFrame yang dibutuhkan, kini saatnya kita melengkapi dashboard tersebut dengan berbagai visualisasi data. Pada bagian awal, kita perlu menambahkan header pada dashboard tersebut. Berikut merupakan kode untuk melakukannya.

```
st.header('Bike Sharing Dashboard :sparkles:')
```

Kode tersebut akan menghasilkan tampilan header seperti di bawah ini

Bike Sharing Dashboard ✨

Menampilkan jumlah total sepeda yang sudah disewa dan total pengguna yang sudah terdaftar

```
col1, col2 = st.columns(2)

with col1:
    total_orders_df = create_sum_rental_df(df)
    total_orders = total_orders_df.total_rental.sum()
    st.metric("Total Bike Sharing", value=total_orders)

with col2:
    registeredUser_df = create_sum_rental_df(df)
    registeredUser = registeredUser_df.registered_user.sum()
    st.metric("Total Registered", value=registeredUser)
```

hasil kode tersebut

Total Bike Sharing

3292679

Total Registered

2672662

Tahap berikutnya adalah menambahkan informasi terkait *pertanyaan bisnis* pada dashboard yang akan kita buat. Pada proyek ini, kita akan menampilkan empat informasi terkait penyewaan sepeda, yaitu Hari Kerja vs Hari Libur, Korelasi Cuaca

dan Penyepeda, serta Korelasi Kelembaban dengan Penyepeda dan Rata-rata Penyewaan Sepeda pada Hari Libur.

Pada contoh proyek ini, kita menampilkan informasi dalam bentuk visualisasi data seperti kode berikut.

Bagian 1, untuk menampilkan Sewa Sepeda Harian Per Musim vs Per Jam.

```
# Menampilkan grafik sewa per musim
st.write("## Jumlah Sewa Sepeda Harian per Musim")
plt.figure(figsize=(8, 6))
plt.bar(season_names, seasonal_data, color=['lightgreen', 'gold', 'orange', 'lightblue'])
plt.xlabel('Musim')
plt.ylabel('Rata-rata Jumlah Sewa Harian')
plt.title('Jumlah Sewa Sepeda Harian per Musim')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(range(4), season_names)
st.pyplot(plt)

seasonal_data_hour = df.groupby('season_hourly')['cnt_hourly'].mean()

# Menampilkan grafik sewa per jam
st.write("## Jumlah Sewa Sepeda per jam")
plt.figure(figsize=(8, 6))
plt.bar(season_names, seasonal_data, color=['lightgreen', 'gold', 'orange', 'lightblue'])
plt.xlabel('Musim')
plt.ylabel('Rata-rata Jumlah Sewa per jam dalam semusim')
plt.title('Jumlah Sewa Sepeda per jam dalam semusim')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(range(4), season_names)
st.pyplot(plt)
```

Bagian 2, untuk pola jumlah sewa sepeda harian berdasarkan bulan dan jam

```
# Menampilkan kedua grafik sewa
st.write("## Pola Jumlah Sewa Sepeda Harian Berdasarkan Bulan dan Jam")
sns.set_style("whitegrid")
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(24, 6))

sns.lineplot(x="mnth_daily", y="cnt_daily", data=df, color='skyblue', ax=ax1)
ax1.set_title("Pola Jumlah Sewa Sepeda Harian Berdasarkan Bulan")
ax1.set_xlabel("Bulan")
ax1.set_ylabel("Jumlah Sewa Sepeda Harian")
ax1.set_xticks(range(1, 13))
```

```

ax1.set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])

sns.lineplot(x="hr", y="cnt_hourly", data=df, color='orange', ax=ax2)
ax2.set_title("Pola Jumlah Sewa Sepeda Harian Berdasarkan Jam")
ax2.set_xlabel("Jam")
ax2.set_ylabel("Jumlah Sewa Sepeda Harian")
ax2.set_xticks(range(24))
ax2.grid(axis='both', linestyle='--', alpha=0.5)
st.pyplot(fig)

```

Bagian 3, untuk Korelasi cuaca terhadap penyewaan sepeda

```

st.write("## Pengaruh Cuaca Terhadap Jumlah Sewa Sepeda Harian")
plt.figure(figsize=(12, 8))
avg_weather =
df.groupby('weather_label')['cnt_daily'].mean().reset_index().sort_values("cnt_daily")
sns.barplot(x="weather_label", y="cnt_daily", data=avg_weather, estimator=sum)
plt.title("Pengaruh Cuaca Terhadap Jumlah Sewa Sepeda Harian", fontsize=16)
plt.xlabel("Cuaca", fontsize=14)
plt.ylabel("Total Jumlah Sewa Sepeda Harian", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.grid(axis='y', linestyle='--')
st.pyplot(plt)

```

Bagian 4, rata-rata perbandingan sewa sepeda setiap hari

```

st.write("## Perbandingan Hari Sewa Sepeda Setiap Hari ")
plt.figure(figsize=(8, 5))
sns.barplot(x='weekday_daily', y='cnt_daily', data=df, palette='Set1')
plt.title('Rata-rata Penyewaan Sepeda Setiap Hari')
plt.xlabel('Hari')
plt.ylabel('Rata-rata Penyewaan')
plt.xticks([0, 1, 2, 3, 4, 5, 6], ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu'])
st.pyplot(plt)

```

Nah, kode tersebut akan menghasilkan tampilan dashboard seperti berikut.

Bike Sharing Dashboard ✨

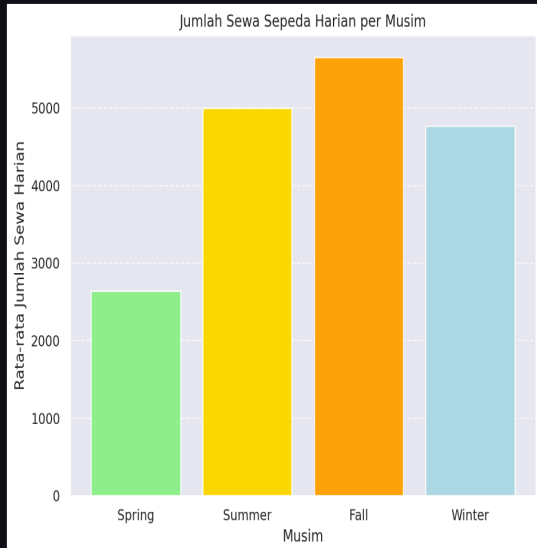
Total Bike Sharing

3292679

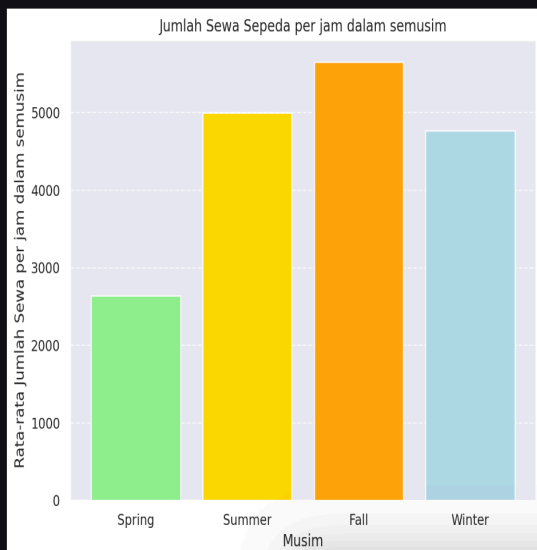
Total Registered

2672662

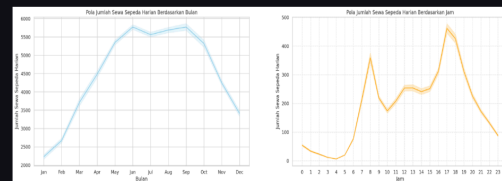
Jumlah Sewa Sepeda Harian per Musim



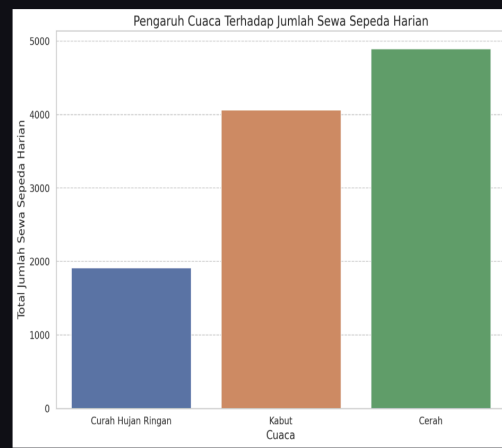
Jumlah Sewa Sepeda per jam



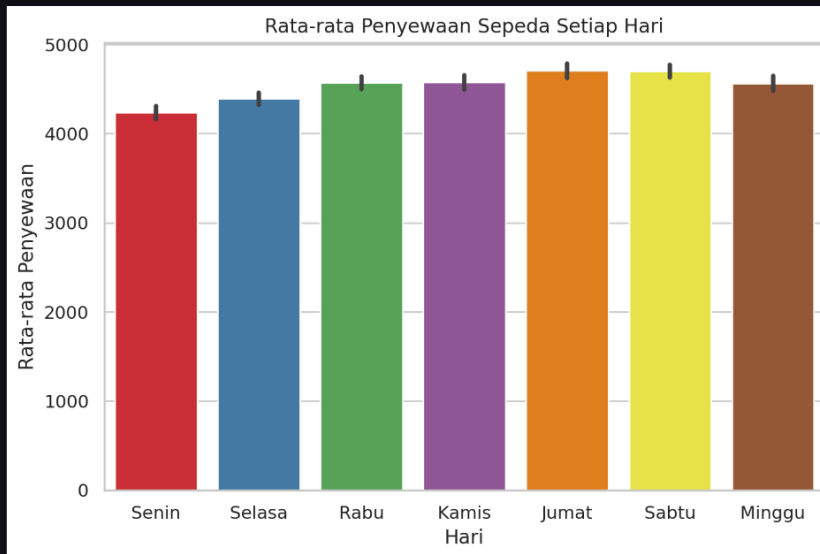
Pola Jumlah Sewa Sepeda Harian Berdasarkan Bulan dan Jam



Pengaruh Cuaca Terhadap Jumlah Sewa Sepeda Harian



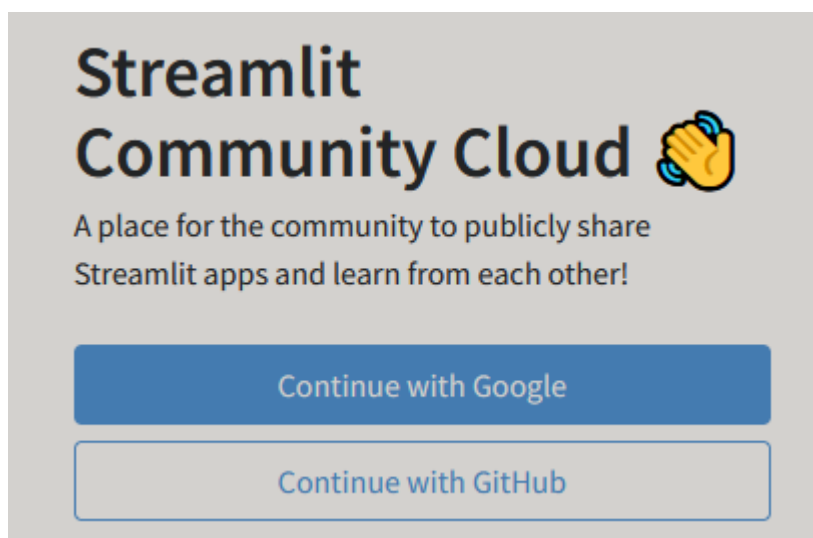
Perbandingan Hari Sewa Sepeda Setiap Hari



Oke, itulah berbagai visualisasi data yang harus kita tampilkan dalam dashboard. Jika seluruh proses di atas berjalan lancar, Anda akan memperoleh tampilan dashboard seperti berikut.

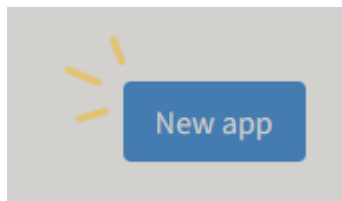
Melakukan Deploy di StreamLit

Pertama-tama, Buka situs web Streamlit Sharing di share.streamlit.io

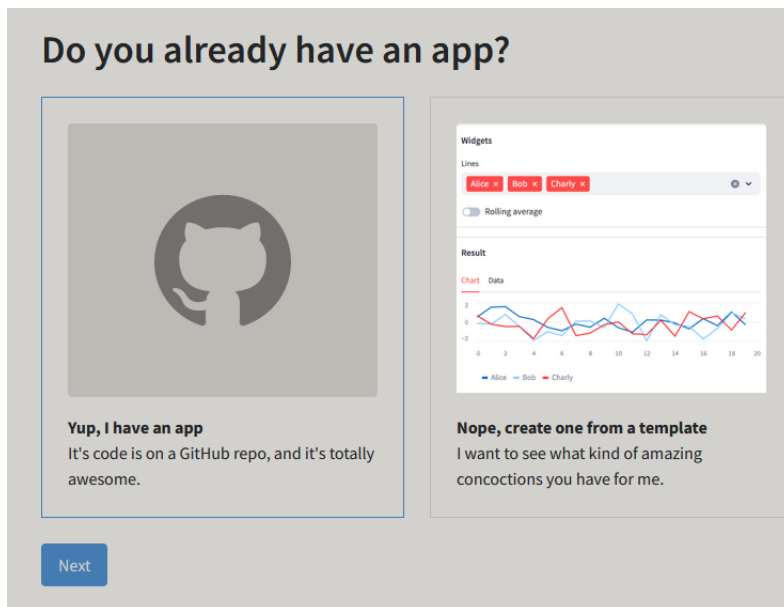


Jika kamu belum memiliki akun, daftarlh untuk membuat akun baru. Jika sudah, masuklah ke akun kamu. Pro Tips: Jangan lupa hubungkan juga ke Githubmu ya.

Setelah masuk, Anda akan melihat tombol "New app".



Klik tombol tersebut dan Anda akan diarahkan ke halaman upload.



Pilih "Yup, I have an app" lalu klik next.

Deploy an app

Repository [Paste GitHub URL](#)

allanbil214/manabebaik

Branch

main

Main file path

dashboard/dashboard.py

App URL (Optional)

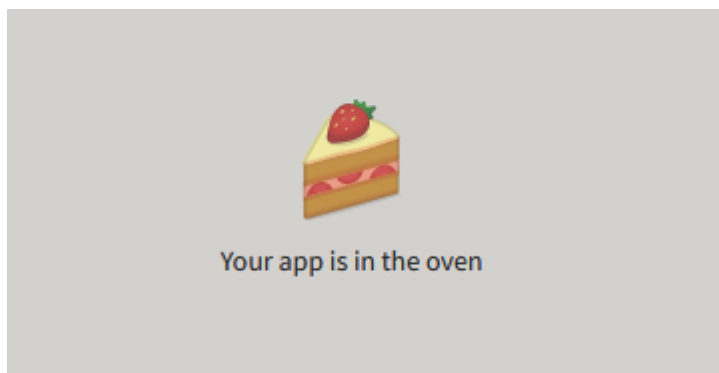
manabebaik2 .streamlit.app

Domain is available

[Advanced settings...](#)

[Deploy!](#)

Lalu disitu diisi dengan Repo githubmu, pastikan path file nya sudah benar, dan kamu bisa rubah URL app nya sesuai dengan keinginanmu.



Voila, aplikasimu sudah dideploy! Tinggal tunggu aplikasinya matang di oven ya.



Setelah selesai di Oven maka aplikasimu akan berjalan, Selamat telah mendeploy app pertamamu di StreamLit!